EXP7

10 August 2026

[1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree

from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix
```

[2]:
```
df = pd.read_excel("Online Retail.xlsx")

# Display first five rows
print(df.head())

# Dataset information
print(df.info())
```

```
  InvoiceNo StockCode                          Description  Quantity  \
0    536365    85123A   WHITE HANGING HEART T-LIGHT HOLDER         6
1    536365     71053                  WHITE METAL LANTERN         6
2    536365    84406B       CREAM CUPID HEARTS COAT HANGER         8
3    536365    84029G  KNITTED UNION FLAG HOT WATER BOTTLE         6
4    536365    84029E       RED WOOLLY HOTTIE WHITE HEART.         6

          InvoiceDate  UnitPrice  CustomerID         Country
0 2010-12-01 08:26:00       2.55     17850.0  United Kingdom
1 2010-12-01 08:26:00       3.39     17850.0  United Kingdom
2 2010-12-01 08:26:00       2.75     17850.0  United Kingdom
3 2010-12-01 08:26:00       3.39     17850.0  United Kingdom
4 2010-12-01 08:26:00       3.39     17850.0  United Kingdom
<class 'pandas.DataFrame'>
RangeIndex: 541909 entries, 0 to 541908
Data columns (total 8 columns):
```

```
 #   Column       Non-Null Count   Dtype
---  ------       --------------   -----
 0   InvoiceNo    541909 non-null  object
 1   StockCode    541909 non-null  object
 2   Description  540455 non-null  object
 3   Quantity     541909 non-null  int64
 4   InvoiceDate  541909 non-null  datetime64[us]
 5   UnitPrice    541909 non-null  float64
 6   CustomerID   406829 non-null  float64
 7   Country      541909 non-null  str
dtypes: datetime64[us](1), float64(2), int64(1), object(3), str(1)
memory usage: 40.0+ MB
None
```

[3]:
```
print("\nDataset Description\n")
print(df.describe())
```

```
Dataset Description

            Quantity                    InvoiceDate      UnitPrice     CustomerID
count  541909.000000                         541909  541909.000000  406829.000000
mean        9.552250  2011-07-04 13:34:57.156386       4.611114   15287.690570
min    -80995.000000          2010-12-01 08:26:00  -11062.060000   12346.000000
25%         1.000000          2011-03-28 11:34:00       1.250000   13953.000000
50%         3.000000          2011-07-19 17:17:00       2.080000   15152.000000
75%        10.000000          2011-10-19 11:27:00       4.130000   16791.000000
max     80995.000000          2011-12-09 12:50:00   38970.000000   18287.000000
std       218.081158                          NaN      96.759853    1713.600303
```

[4]:
```
print("\nDataset Shape")
print(df.shape)
```

```
Dataset Shape
(541909, 8)
```

[5]:
```
print("\nMissing Values")
print(df.isnull().sum())
```

```
Missing Values
InvoiceNo          0
StockCode          0
Description     1454
Quantity           0
InvoiceDate        0
UnitPrice          0
CustomerID     135080
```

```
Country              0
dtype: int64
```

[6]:
```python
# Remove missing values
df.dropna(inplace=True)
```

[7]:
```python
# Remove duplicate records
df.drop_duplicates(inplace=True)
```

[8]:
```python
# Keep only positive Quantity
df = df[df["Quantity"] > 0]
```

[9]:
```python
# Keep only positive UnitPrice
df = df[df["UnitPrice"] > 0]
```

[10]:
```python
# Remove cancelled invoices
df = df[~df["InvoiceNo"].astype(str).str.startswith("C")]
```

[11]:
```python
df["TotalAmount"] = df["Quantity"] * df["UnitPrice"]
```

[12]:
```python
median_value = df["TotalAmount"].median()
df["CustomerSegment"] = np.where(
    df["TotalAmount"] >= median_value,
    "High",
    "Low"
)
```

[13]:
```python
encoder = LabelEncoder()

df["Country"] = encoder.fit_transform(df["Country"])

df["StockCode"] = df["StockCode"].astype(str)
df["StockCode"] = encoder.fit_transform(df["StockCode"])
df["CustomerSegment"] = encoder.fit_transform(df["CustomerSegment"])
```

[14]:
```python
temp_df = df.copy()

X = temp_df[[
    "Quantity",
    "UnitPrice",
    "Country",
    "StockCode"
]]

# Ensure all columns in X are numeric (float) for the model
X = X.astype(float)

# Align y with the potentially modified X after dropping rows
```

```
y = df.loc[temp_df.index, "CustomerSegment"]
```

[15]:
```
X_train, X_test, y_train, y_test = train_test_split(

    X,

    y,

    test_size=0.20,

    random_state=42,

    stratify=y
)
```

[16]:
```
model = DecisionTreeClassifier(

    criterion="entropy",

    random_state=42
)

model.fit(X_train, y_train)
```

[16]:
```
DecisionTreeClassifier(criterion='entropy', random_state=42)
```

[17]:
```
prediction = model.predict(X_test)
```

[18]:
```
print("\nAccuracy")
print(accuracy_score(y_test, prediction))
print("\nClassification Report")
print(classification_report(y_test, prediction))
print("\nConfusion Matrix")
print(confusion_matrix(y_test, prediction))
```

```
Accuracy
0.9999745349444226

Classification Report
              precision    recall  f1-score   support

           0       1.00      1.00      1.00     39270
           1       1.00      1.00      1.00     39269

    accuracy                           1.00     78539
   macro avg       1.00      1.00      1.00     78539
weighted avg       1.00      1.00      1.00     78539
```

```
Confusion Matrix
[[39270     0]
 [    2 39267]]
```

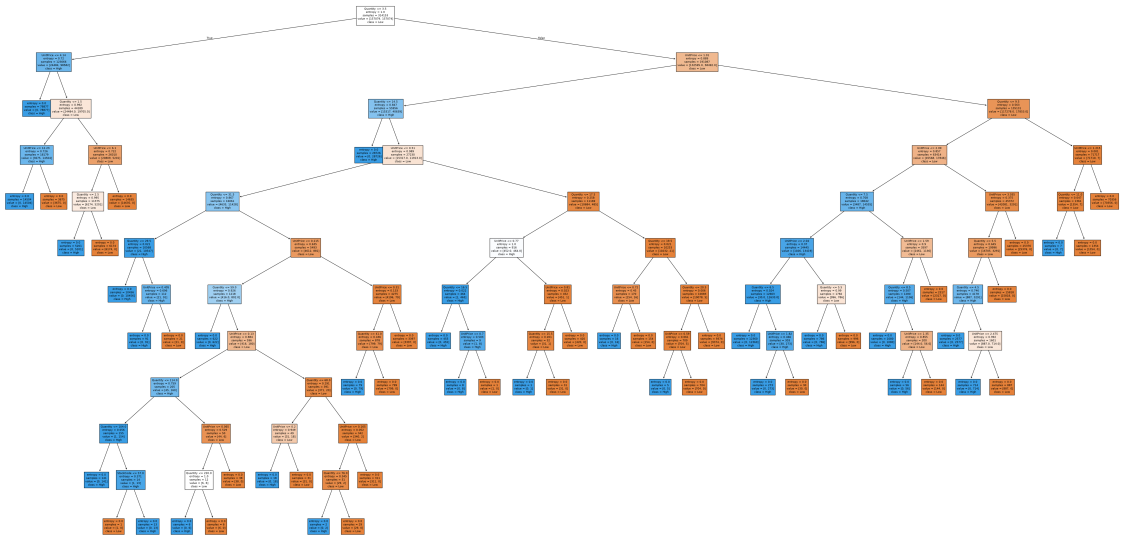
[19]:
```
importance = pd.DataFrame({

    "Feature": X.columns,

    "Importance": model.feature_importances_
})
print("\nFeature Importance")
print(importance)
```

```
Feature Importance
     Feature  Importance
0   Quantity    0.485063
1  UnitPrice    0.514920
2    Country    0.000000
3  StockCode    0.000017
```

[20]:
```
plt.figure(figsize=(60,30))
plot_tree(
    model,
    feature_names=X.columns,
    class_names=["Low","High"],
    filled=True,
    fontsize=8
)
plt.show()
```



[21]:
```
importance = importance.sort_values(
    by="Importance",
    ascending=False
)
plt.figure(figsize=(8,5))
plt.bar(
    importance["Feature"],
    importance["Importance"]
)
plt.xlabel("Features")
plt.ylabel("Importance")
plt.title("Feature Importance")
plt.show()
```

